

[Edit](#)

Create Branches and Merge Changes Safely

Challenge Overview

Understand the scenario

You are a Developer for Hexelo, an organization that needs to create, switch, and merge branches. In this Challenge Lab, you will build features on separate branches and combine them into main while handling conflicts. First, you will create and switch to new branches, and then you will make changes in a feature branch. Next, you will merge branches back to main, and then you will resolve simple merge conflicts. Next, you will use `git log --graph` to visualize history, and then you will create and merge branches. Finally, you will handle a merge conflict manually.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

🔗 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Configure your environment and starter project

Hints Enabled

No Yes

- Sign in to the virtual machine as **LabUser** using **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, Password, and GitHub Account URL into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

GitHub URL

- Open **Microsoft Edge**, go to <https://github.com>, and then sign in as **<Name>** using **<GithubPW>** as the password.



Expand this hint for guidance on signing in to a GitHub account.



- On the Windows taskbar, in Search, enter **Edge**, and then select **Microsoft Edge** to open a new browser.
- In the Address bar, enter <https://github.com>, and then press **Enter**.
- On the GitHub home page, in the upper right-hand corner, select **Sign in**.
- On the *Sign in to GitHub* page, in Username or email address, sign in as **<Name>** using **<GithubPW>** as the password.
- If presented with the *Save password* dialog, select **Never**.
- On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.

- Open a new **Microsoft Edge** browser tab, go to <https://git-scm.com/download/win>, download and install the latest version of **Git** by using the default options, and then launch **Git Bash**.

 Expand this hint for guidance on installing Git.

- In Microsoft Edge, open a new tab.
- In the Address bar, enter <https://git-scm.com/download/win>, and then press **Enter**.
- On the **git** Downloads page, in *Downloads for Windows*, select **Click here to download**.
- In the *Downloads notification* dialog, select **Open file**.
- In the *User Account Control - Do you want to allow this app to make changes to your device?* dialog, select **Yes**.
- On the Git Setup page, ensure that *Only show new options* is selected, select **Next** (if necessary), and then select **Install**.
- On the *Completing the Git Setup Wizard* page, select the **Launch Git Bash** checkbox, and then select **Finish**.
- If prompted, in the *Select an app to open this .html file* dialog, select **Always**, and then review and close the **Release Notes** browser tab.

 Want to learn more? Review the documentation on [installing git](#).

- Verify the installation by running the following command in the **Git Bash** Terminal:

```
git --version
```

 The results should display *git version 2.52.0.windows.1* or later.

Always verify Git is installed before running Git Bash commands.

 Want to learn more? Review the documentation on the [Git Bash Terminal](#).

- Set your Git Identity by using the [git config](#) command using your GitHub **username** and **email address**.

💡 Expand this hint for guidance on setting your Git identity. ^

- In the Git Bash Terminal, run the following commands to set your Git identity:

```
git config --global user.name "<Name>"
```

```
git config --global user.email "<GithubUN>"
```

💡 Want to learn more? Review the documentation on [setting your Git Identity](#).

💡 The `--global` Git identity can be changed as many times as you like (each change overwrites the previous one), and you can always override it for individual repositories using `--local`.

- Confirm your settings by using the `T git config` command.

💡 Expand this hint for guidance on confirming your settings. ^

- Run the following command to confirm your settings:

```
git config --list
```

💡 Want to learn more? Review the documentation on [confirming your settings](#).

Create a local repository

- In the Git Bash Terminal, create a project folder named `T lab-git-intro{LAB_INSTANCE_ID}` in the `T D:\LabFiles` folder and change to the new folder by running the following command:

```
mkdir /d/LabFiles/lab-git-intro{LAB_INSTANCE_ID} && cd /d/LabFiles/lab-git-intro
```

💡 Want to learn more? Review the following command documentation:

- `cd`
- `mkdir`

- Initialize the Git repository by running the following command:

```
git init
```

 The results should look like the following screenshot:



```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro
$ git init
Initialized empty Git repository in D:/LabFiles/lab-git-intro/.git/
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (master)
$
```

 Want to learn more? Review the documentation on [initializing a Git repo](#).

 The .git folder inside a repository is where Git stores all its metadata—including your commit history, branches, and configuration. Deleting this folder will "un-Git" your project.

- Rename your branch from *master* to `[T] main` by running the following `[T] git branch` command:

```
git branch -M main
```

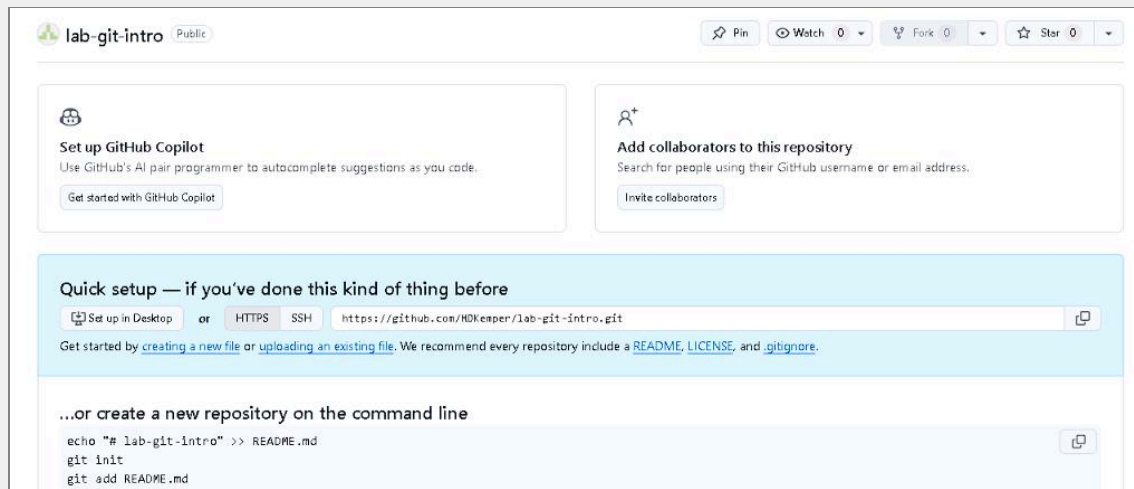
 Want to learn more? Review the documentation on [renaming a branch](#).

Create a GitHub repository

- Switch to the **GitHub** browser tab, and then if needed, sign in as `[T] <Name>` using `[T] <GithubPW>` as the password.
- Create a GitHub repository named `[T] lab-git-intro{LAB_INSTANCE_ID}`.

💡 Expand this hint for guidance on creating a GitHub repository.

- On the GitHub Dashboard Home page, in Create your first project, select **Create repository**.
- On the New repository page, in Create a new repository, in Repository name, enter `lab-git-intro{LAB_INSTANCE_ID}`, and then select **Create repository**.
- Your new repository page should look like the following screenshot:



💡 Repositories on GitHub can be public or private. Public repos are visible to everyone, while private repos restrict visibility to collaborators you choose—great for proprietary or in-progress work.

💡 Want to learn more? Review the documentation on [creating a GitHub repository](#).

- In the Git Bash Terminal, connect your local repository to your remote repository using the URL `<GithubURL>/lab-git-intro{LAB_INSTANCE_ID}.git` and by running the following `git remote` command:

```
git remote add origin <GithubURL>/lab-git-intro{LAB_INSTANCE_ID}.git
```

💡 Want to learn more? Review the documentation on the [git remote](#) command.

Check your work

Verify

Commit changes and create a new branch

Hints Enabled

No Yes

- Create your first commit by using the `echo`, `git add`, and `git commit` commands.
 - Create a `README.md` file with the text `"# lab-git-intro"`.
 - Stage the `README.md` file using `git add`.
 - Commit the changes by using `git commit` with the text `"Initial commit"`.

💡 Expand this hint for guidance on creating a commit.

- In the Git Bash Terminal, enter `echo "# lab-git-intro" > README.md && git add README.md && git commit -m "Initial commit"`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$ echo "# lab-git-intro" > README.md && git add README.md && git commit -m "Initial commit"
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
[main e0edb97] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 README.md

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
```

📄 You can chain commands like `echo > file && git add file && git commit -m "message"` to streamline work, but beginners should do each one step-by-step until comfortable.

- Create a second commit by running the following command:

```
echo "Hello, Git World!" > hello.txt && git add hello.txt && git commit -m "Add hello.txt"
```

- Ensure you have the remote repository set by using the `git remote` command.

💡 Expand this hint for guidance on verifying the remote repository. ^

- In the Git Bash Terminal, enter `T git remote -v`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$ git remote -v
origin https://github.com/MDKemper/lab-git-intro (fetch)
origin https://github.com/MDKemper/lab-git-intro (push)

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$
```

- Use the `T git push` command to push to GitHub, sign in to GitHub, and then **Authorize git-ecosystem**.

💡 Expand this hint for guidance on pushing to GitHub. ^

- Run the following command to push to GitHub:

```
git push -u origin main
```

- When prompted, sign in to GitHub by using the **Sign in with your browser** option.
- If prompted, on the *Authorize Git Credential Manager* page, select **Authorize git-ecosystem**.
- Close the **Git Credential Manager** browser tab.

💡 Want to learn more? Review the documentation on [pushing to GitHub](#).

📄 **Connect and Push to GitHub:** Once you've connected your local repo to GitHub, you can use `git push` for future updates without `-u origin main`—Git remembers the upstream branch after the first push.

If your push fails, it's often because of a mismatch or missing remote—use `git remote -v` to confirm.

- Create a new branch named `T feature-welcome-message` by using the `T git branch` command.

💡 Expand this hint for guidance on creating a new branch. ^

- In the Git Bash Terminal, enter `T git branch feature-welcome-message`, and then press **Enter**.



Want to learn more? Review the documentation on [creating a new branch](#).

Check your work

Verify

Merge the new branch into main

Hints Enabled

No Yes

- Switch to the `feature-welcome-message` branch by using the `git switch` command.

 Expand this hint for guidance on switching to a new branch. 

- In the Git Bash Terminal, enter `git switch feature-welcome-message`, and then press **Enter**.

 Want to learn more? Review the documentation on [switching to a new branch](#).

 Use descriptive names for branches, like `feature-welcome-message`, so collaborators understand its purpose.

- Add a new line to the file named `hello.txt` with the text `"Welcome to branching in Git!"` by using the `echo` command.

 Expand this hint for guidance on adding a new line to a file. 

- In the Git Bash Terminal, enter `echo "Welcome to branching in Git!" >> hello.txt`, and then press **Enter**.

 Confirm the text has been added to the file by running the command: `cat hello.txt`.

- Stage the change by using the `git add` command with the file `hello.txt`.

 Expand this hint for guidance on staging a change.

- In the Git Bash Terminal, enter `T git add hello.txt`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (feature-welcome-message)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be replaced by CRLF the next time Git touches it
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (feature-welcome-message)
$
```

 Want to learn more? Review the documentation on [staging a change](#).

- Commit the change by using the `T git commit` command with the message `T "Added welcome message to hello.txt"`.

 Expand this hint for guidance on committing a change.

- In the Git Bash Terminal, enter `T git commit -m "Added welcome message to hello.txt"`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (feature-welcome-message)
$ git commit -m "Added welcome message to hello.txt"
[feature-welcome-message 4b371d9] Added welcome message to hello.txt
1 file changed, 1 insertion(+)
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (feature-welcome-message)
$ |
```

 Want to learn more? Review the documentation on [committing a change](#). Always commit before switching branches—uncommitted changes stay in your working directory and may cause confusion.

- Switch back to the `T main` branch by using the `T git switch` command.

💡 Expand this hint for guidance on switching back to main.

- In the Git Bash Terminal, enter `git switch main`, and then press **Enter**.

```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-intro (feature-welcome-message)
$ git checkout main
D      lab-git-intro-2.zip
D      lab-git-intro.zip
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-intro (main)
$ |
```

- Merge the `feature-welcome-message` branch by using the `git merge` command.

💡 Expand this hint for guidance on merging the feature branch.

- In the Git Bash Terminal, enter `git merge feature-welcome-message`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$ git merge feature-welcome-message
Updating 4dce812..4b371d9
Fast-forward
 hello.txt | 1 +
 1 file changed, 1 insertion(+)

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$ |
```

💡 Want to learn more? Review the documentation on [merging the feature branch](#).

📄 If successful, you'll see output like:

```
Updating 2f101a6..00835c7 Fast-forward hello.txt | 1 +
```

- Delete the `feature-welcome-message` branch by using the `git branch` command.

💡 Expand this hint for guidance on deleting a branch. ^

- In the Git Bash Terminal, enter `T` `git branch -d feature-welcome-message`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$ git branch -d feature-welcome-message
Deleted branch feature-welcome-message (was 4b371d9).

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$
```

💡 Want to learn more? Review the documentation on [deleting a branch](#).

📄 After a successful merge, delete your feature branch locally with `-d`, but use `-D` if it wasn't merged and you want to force deletion.

Check your work

Verify

View history and differences

Hints Enabled

No Yes

- View the commit history by using the `git log` command.

💡 Expand this hint for guidance on viewing the commit history. ^

- In the Git Bash Terminal, enter `git log --oneline`, and then press **Enter**.

💡 Want to learn more? Review the documentation on [viewing the commit history](#).

📄 `git log --oneline` is a cleaner way to review commit history; use it frequently to understand your project timeline.

- View the file difference to see what changed recently by using the `git diff` command.

💡 Expand this hint for guidance on viewing the file difference. ^

- In the Git Bash Terminal, enter `git diff HEAD~1 HEAD`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$ git diff HEAD~1 HEAD
diff --git a/hello.txt b/hello.txt
index c671047..00de5b5 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1,2 @@
 Hello, Git world!
+Welcome to branching in Git!

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$
```

💡 Want to learn more? Review the documentation on [viewing the file difference](#).

📄 Use `HEAD~1` to compare with the previous commit, but you can also use specific commit hashes for more precise diffs. ^

Use `git log --graph` (as mentioned in the lab description) to visually understand how branches diverged and merged.

Color can help: Add `--color` to many Git commands for better visual clarity (`git log --oneline --color`).

- Review the contents of the git reference logs by using the `git reflog` command.



Want to learn more? Review the documentation on the [git reflog](#) command.



Look for entries like *merge feature-welcome-message...* or *checkout: moving from feature-welcome-message to*.

- Repeat the previous command and save the output to a file named `reflog.txt` by running the following command:

```
git reflog > reflog.txt
```

- Stage and commit the `reflog.txt` file by running the following command:

```
git add reflog.txt && git commit -m "Added reflog.txt to the repo"
```

Check your work

Verify

Resolve a merge conflict

Hints Enabled

No Yes

- Create and switch to a new branch named `conflicting-branch` by using the `git switch` command.

 Expand this hint for guidance on creating and switching to a new branch. 

- In the Git Bash Terminal, enter `git switch -c conflicting-branch`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (main)
$ git checkout -b conflicting-branch
Switched to a new branch 'conflicting-branch'

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (conflicting-branch)
$ |
```

 Want to learn more? Review the documentation on the [git switch](#) command.

 It is common practice to use a different branch name (like `conflicting-branch`) to avoid affecting other work.

- To create a conflict with the main branch, modify the `hello.txt` file with the text `This line will conflict with main` by using the `echo` command. This will overwrite the file contents instead of appending data.

 Expand this hint for guidance on overwriting file contents. 

- In the Git Bash Terminal, enter `echo "This line will conflict with main" > hello.txt`, and then press **Enter**.

- Stage and commit the change by using the `git add` and `git commit` commands with the message `"Conflicting change in conflicting-branch"`.

💡 Expand this hint for guidance on staging and committing a change. ^

- In the Git Bash Terminal, enter `T git add hello.txt`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (conflicting-branch)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be replaced by CRLF the next time Git touches it.

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (conflicting-branch)
$ |
```

- In the Git Bash Terminal, enter `T git commit -m "Conflicting change in conflicting-branch"`, and then press **Enter**.

```
Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (conflicting-branch)
$ git commit -m "Conflicting change in conflicting-branch"
[conflicting-branch 5e76f19] Conflicting change in conflicting-branch
1 file changed, 1 insertion(+), 2 deletions(-)

Admin@DESKTOP-BR9K513 MINGW64 /d/LabFiles/lab-git-intro (conflicting-branch)
$
```

- Switch back to the `T main` branch by using the `T git switch` command.
- Overwrite the `T hello.txt` file with the text `T "This line is from main and will also conflict"` by running the following command:

```
echo "This line is from main and will also conflict" > hello.txt
```

- Stage the change by using the `T git add` command with the file `T hello.txt`.
- Commit the change by using the `T git commit -m` command with the message `T "Conflicting change in main"`.
- Attempt to merge the `T conflicting-branch` branch by using the `T git merge` command.

💡 Expand this hint for guidance on merging a branch. ^

- In the Git Bash Terminal, enter `T git merge conflicting-branch`, and then press **Enter**.

```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-intro (main)
$ git merge conflicting-branch
Auto-merging hello.txt
CONFLICT (content): Merge conflict in hello.txt
Automatic merge failed; fix conflicts and then commit the result.

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-intro (main|MERGING)
$
```



Want to learn more? Review the documentation on [merging a branch](#).



You will see an error:

Auto-merging hello.txt

CONFLICT (content): Merge conflict in hello.txt

Automatic merge failed; fix conflicts and then commit the result.

- Open the `hello.txt` file and modify it by adding the text `Final version after resolving conflict.`



Expand this hint for guidance on modifying a file during a merge conflict.



- In the Git Bash Terminal, open the `hello.txt` file by running:

```
notepad hello.txt
```

- In the file, locate the conflict markers:

```
<<<<<< HEAD
=====
>>>>>> conflicting-branch
```

- Review both versions of the text.
- In the `hello.txt` file, press **Ctrl + A**, press **Delete**, and then enter the text `Final version after resolving conflict.`



- Save the file and close Notepad.



Don't panic when you see conflict markers like `<<<<<<`, `=====`, and `>>>>>>`—they show exactly where Git needs help.



Only remove conflict markers, not the actual data you want to keep—make sure the final version has only the resolved lines.

- Stage the resolved `hello.txt` file by using the `git add` command.

💡 Expand this hint for guidance on staging the resolved file.

- In the Git Bash Terminal, run the following command:

```
git add hello.txt
```

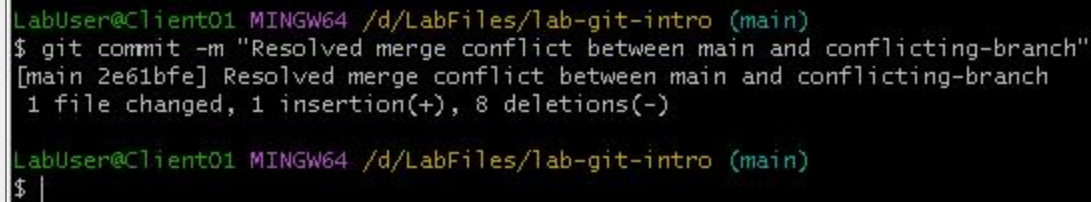
- Verify the file is staged by running:

```
git status
```

- The output should list **hello.txt** under *Changes to be committed*.

- Commit the merge resolution by using the `git commit -m` command with the message "Resolved merge conflict between main and conflicting-branch".

📄 The results should look like the following screenshot:



```
LabUser@Client01 MINGW64 /d/LabFiles/lab-git-intro (main)
$ git commit -m "Resolved merge conflict between main and conflicting-branch"
[main 2e61bfe] Resolved merge conflict between main and conflicting-branch
1 file changed, 1 insertion(+), 8 deletions(-)

LabUser@Client01 MINGW64 /d/LabFiles/lab-git-intro (main)
$
```

💡 Want to learn more? Review the documentation on [staging and committing a resolution](#).

📄 After resolving, always run `git add` and then `git commit` to finish the merge; otherwise, your repository remains in a conflicted state.

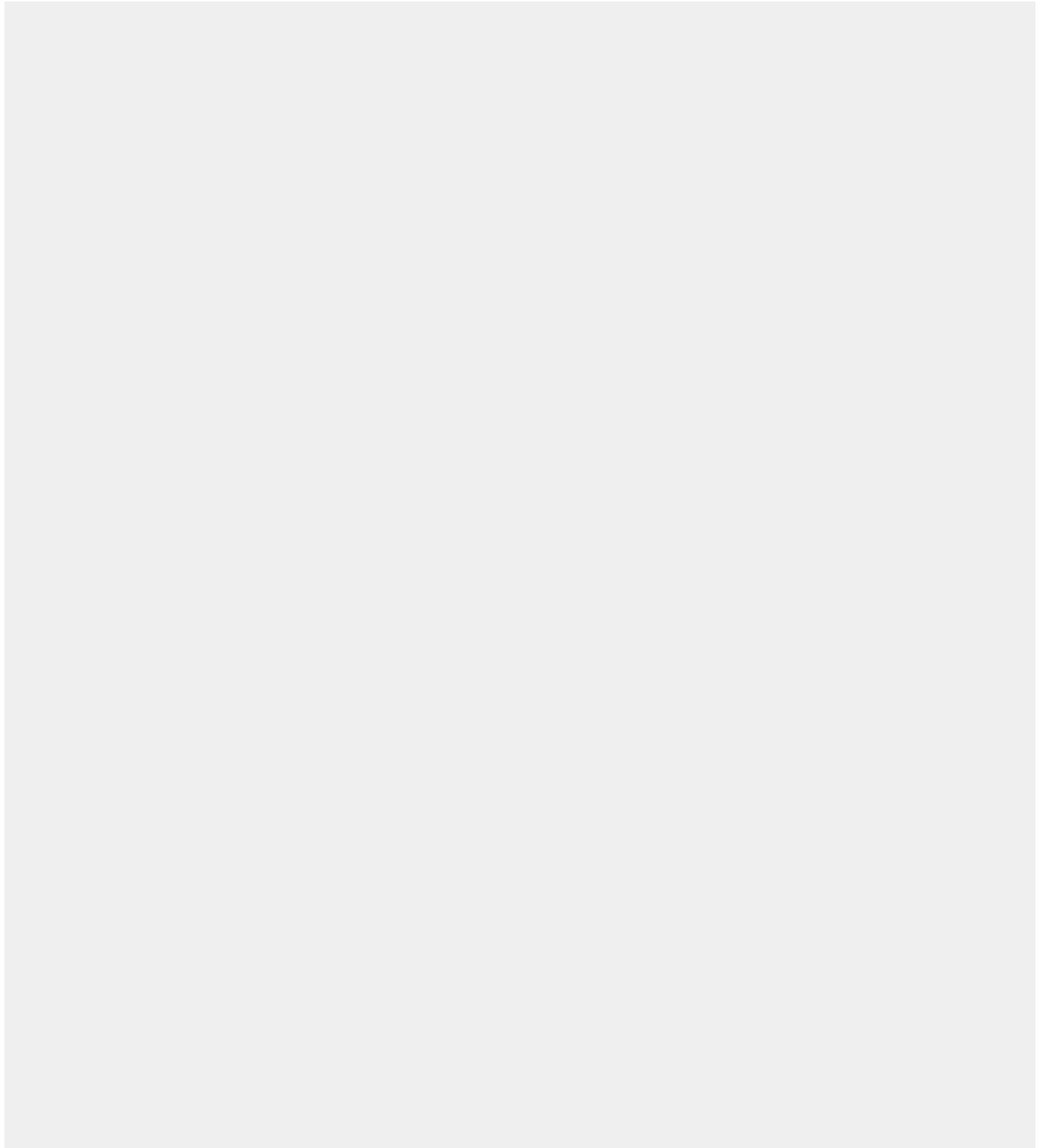
You have resolved your first Git merge conflict.

- Delete the `conflicting-branch` branch by using the `git branch -d` command.
- Use the `git push origin main` command to push to GitHub, sign in to GitHub, if necessary, and then **Authorize git-ecosystem**.

Check your work



Verify



Summary

Congratulations, you have completed the **Create Branches and Merge Changes Safely** Challenge Lab.

You have accomplished the following:

- Configured your environment and starter project.
- Committed changes and created a new branch.
- Merged the new branch into main.
- Viewed history and differences.
- Resolved a merge conflict.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Track Local Changes and Manage Commits [Guided]
- Create Branches and Merge Changes Safely [Guided]
- Collaborate Remotely and Use Pull Requests [Guided]
- Build a Real Project by Using Git Workflows [Advanced]

- Write Code with GitHub Copilot Suggestions [Guided]
- Control Git Behavior with Config and Ignore Files [Guided]
- Rebase, Stash, and Apply Commits Precisely [Guided]
- Manage Tasks by Using GitHub Issues and Boards [Guided]
- Automate Testing and CI with GitHub Actions [Guided]
- Review and Refactor Code with Copilot [Guided]
- Implement Release Management and Versioning in GitHub [Expert]

